Students Academic Repository

Cloud-Based Student Project Repository & Retrieval System

Site Documentation

Comprehensive overview of functionalities, screens, architecture, and code locations

Live URL: https://miniproject-one-steel.vercel.app/

Technology: Next.js 16 · React 19 · Supabase · Tailwind CSS 4

Date: September 2026

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Directory Structure & Code Locations
5. Database Schema
6. Authentication & Authorization
7. Screens & Pages
    a. Landing / Home Page
    b. Registration Page
    c. Login Page
    d. Forgot Password
    e. Reset Password
    f. Student Dashboard
    g. Upload Project
    h. Upload History
    i. Bookmarks
    j. Profile Settings
    k. Projects Search / Browse
    l. Project Detail
    m. Admin Dashboard
    n. Admin Projects Management
    o. Admin Users Management
    p. Admin Statistics
8. Communication & Data Flow
9. Notification System
10. File Storage & Downloads
11. PWA (Progressive Web App)
12. Security Measures
13. Deployment
14. Shared Components

1. Project Overview

The **Students Academic Repository (SAR)** is a cloud-based web application designed to serve as a centralized digital archive for student academic projects. It allows students to upload, manage, and showcase their research work, while administrators review and curate submissions.

> **Key Objectives:**
>
> - Provide a secure, searchable repository for student projects across all departments
> - Allow students to upload project reports (PDF), source code (ZIP), GitHub links, live URLs, and images
> - Implement an admin-reviewed approval workflow before projects are published
> - Enable advanced search by title, author, department, academic year, keywords, and project type
> - Track downloads, views, and provide analytics for administrators
> - Support Progressive Web App (PWA) installation on mobile and desktop

User Roles

Role	Capabilities
Student	Register, login, upload projects, search/browse approved projects, download files, bookmark projects, mark projects as helpful, view profile, view notifications
Admin	All student capabilities + approve/reject projects, view all projects (any status), manage users (search, deactivate), view platform statistics with charts, delete projects
Guest	View the landing page (locked project preview), register an account. Cannot search, browse, or download projects.

2. Technology Stack

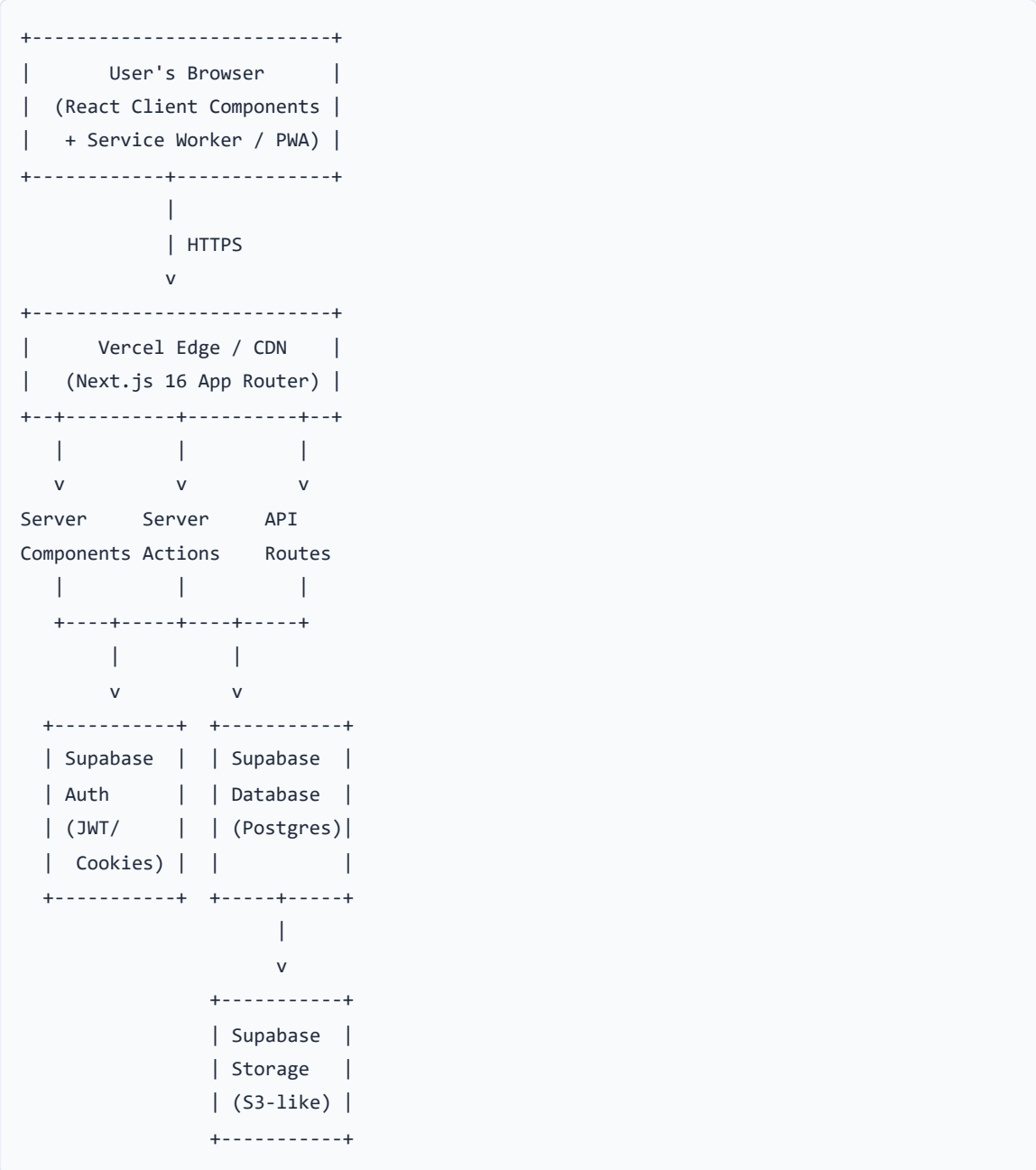
Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	16.3.0	Full-stack React framework with server-side rendering, routing, API routes

UI Library	React	19.2.8	Component-based user interface
Styling	Tailwind CSS	4.x	Utility-first CSS with dark mode support
Icons	Lucide React	1.28.0	Modern SVG icon library
Charts	Recharts	3.10.1	Bar charts for admin statistics
Database	Supabase (PostgreSQL)	--	Cloud database with Row Level Security
Authentication	Supabase Auth	--	Email/password auth with email confirmation
File Storage	Supabase Storage	--	Private bucket for PDFs, ZIPs, images
Server SDK	@supabase/ssr + @supabase/supabase-js	0.12.4 / 2.112.0	Server-side Supabase client with cookie-based auth
Language	TypeScript	5.x	Type-safe JavaScript
Hosting	Vercel	--	Serverless deployment, CDN, edge functions

3. System Architecture

The application follows a modern full-stack architecture using Next.js App Router with Server Components, Server Actions, and API Routes.

Architecture Diagram (Logical)



Data Flow Summary

1. **Authentication:** Supabase Auth issues JWTs stored in HTTP-only cookies via `@supabase/ssr`. Server Components read cookies to identify the current user.
2. **Server Components:** Pages like the Home, Dashboard, and Bookmarks render on the server, directly querying Supabase via the service-role admin client.
3. **Server Actions:** Mutations (upload, approve, delete, bookmark, etc.) use `'use server'` functions in `lib/actions/`. These run server-side and are called from Client Components.
4. **API Routes:** The download endpoint (`/api/download/[id]`) generates signed URLs for private files and redirects the browser.
5. **Client Components:** Interactive pages (search, admin panels, profile) use `'use client'` with `useState` / `useEffect` and call Server Actions for data.

4. Directory Structure & Code Locations

```
miniproject/
+-- app/                          # Next.js App Router (pages & layouts)
|   +-- page.tsx                  # Landing / Home page
|   +-- layout.tsx                # Root layout (fonts, theme, auth, PWA, header)
|   +-- globals.css               # Global styles + Tailwind
|   +-- loading.tsx               # Root loading skeleton
|   +-- error.tsx                 # Global error boundary
|   +-- not-found.tsx             # 404 page
|   +-- login/page.tsx            # Login screen (student / admin toggle)
|   +-- register/page.tsx         # Registration screen
|   +-- forgot-password/page.tsx  # Forgot password screen
|   +-- reset-password/page.tsx   # Reset password screen
|   +-- dashboard/                # Student dashboard area
|   |   +-- layout.tsx            # Dashboard sidebar layout
|   |   +-- page.tsx              # Student dashboard home
|   |   +-- upload/page.tsx       # Upload new project
|   |   +-- history/page.tsx      # Upload history table
|   |   +-- bookmarks/page.tsx    # Bookmarked projects
|   |   +-- profile/page.tsx      # Profile settings
|   +-- admin/                    # Admin area
|   |   +-- layout.tsx            # Admin sidebar layout
|   |   +-- page.tsx              # Admin dashboard (pending queue)
|   |   +-- projects/page.tsx     # All projects management
|   |   +-- users/page.tsx        # User management
|   |   +-- statistics/page.tsx   # Analytics with charts
|   +-- projects/                 # Public project browsing
|   |   +-- page.tsx              # Search / filter projects
|   |   +-- [id]/page.tsx         # Project detail view
|   +-- api/
|       +-- download/[id]/route.ts # File download API (signed URLs)
|       +-- logout/route.ts       # Logout API
|       +-- session/route.ts      # Session refresh API
+-- components/                   # Shared React components
|   +-- Header.tsx                # Global navigation header
|   +-- DashboardSidebar.tsx      # Sidebar for student/admin dashboards
|   +-- AuthProvider.tsx          # Auth context (user state, logout)
|   +-- ThemeProvider.tsx         # Dark/light theme context
|   +-- PWAProvider.tsx           # PWA service worker registration
|   +-- ProjectCard.tsx           # Project card (grid view)
|   +-- ProjectDetailClient.tsx   # Full project detail view
|   +-- ProjectStatusBadge.tsx    # Status badge (Pending/Approved/Rejected)
|   +-- ProjectArtifactBadges.tsx # Artifact badges (PDF/ZIP/GitHub/Live/Images)
|   +-- NotificationBell.tsx      # Notification bell with unread count
|   +-- LockedProjectCard.tsx     # Blurred placeholder for guests
|   +-- UnbookmarkButton.tsx      # Remove bookmark button
|   +-- GitHubRepoViewer.tsx      # GitHub repository tree viewer
+-- lib/                          # Business logic & utilities
|   +-- auth.ts                   # getCurrentUser, requireAuth, requireAdmin
|   +-- types.ts                  # TypeScript type definitions
```

```
|   +-- constants.ts               # Departments, programmes, levels, keywords
|   +-- supabase-client.ts         # Browser-side Supabase client
|   +-- supabase-server.ts         # Server-side Supabase clients (admin + auth)
|   +-- actions/                   # Server Actions
|       +-- auth.ts                # createUserProfile, updateProfile, listUsers, etc.
|       +-- projects.ts            # CRUD for projects, bookmarks, downloads, stats
|       +-- notifications.ts       # Create/read/mark notifications
|       +-- github.ts              # Fetch GitHub repo tree via API
+-- supabase/                      # Database schemas
|   +-- schema.sql                 # Main schema (tables, RLS, indexes)
|   +-- atomic_counters.sql        # Atomic increment RPC functions
|   +-- project_types_and_media.sql# Schema additions for project types/images
|   +-- avatars_bucket.sql         # Avatar storage bucket setup
+-- public/                        # Static assets
|   +-- logo.png                   # Application logo
|   +-- manifest.json              # PWA manifest
|   +-- sw.js                      # Service worker
|   +-- icons/                     # PWA icons (192x192, 512x512)
+-- package.json                   # Dependencies & scripts
+-- next.config.ts                 # Next.js configuration
+-- tsconfig.json                  # TypeScript config
```

5. Database Schema

The database is hosted on Supabase (PostgreSQL). All tables use Row Level Security (RLS). The application uses a service-role key for server-side operations, bypassing RLS. File: `supabase/schema.sql`

5.1 Users Table

Column	Type	Description
`userId`	UUID (PK)	References `auth.users(id)`, cascades on delete
`fullName`	TEXT	User's full legal name
`email`	TEXT (unique)	Email address
`role`	TEXT	'student' or 'admin' (CHECK constraint)
`department`	TEXT	Academic department
`indexNumber`	TEXT	Student index number
`programme`	TEXT	BSc, MSc, PhD, BEng, BTech, MBA
`levelYear`	TEXT	100, 200, 300, 400, 500, 600
`contact`	TEXT	10-digit phone number
`active`	BOOLEAN	Account status (deactivation by admin)
`dateCreated`	TIMESTAMPTZ	Registration timestamp

5.2 Projects Table

Column	Type	Description
`projectId`	UUID (PK)	Auto-generated project identifier
`title`	TEXT	Project title
`authorName`	TEXT	Author's name
`department`	TEXT	Department the project belongs to
`academicYear`	TEXT	Year of submission (e.g. "2025")
`abstract`	TEXT	Project abstract/description

`keywords`	TEXT[]	Array of keyword tags
`projectType`	TEXT	Research Paper / Software / Website / Design / Other
`pdfUrl / pdfPath`	TEXT	Download API URL and storage path for the PDF
`githubUrl`	TEXT	GitHub repository URL
`sourceCodeZipUrl / sourceCodeZipPath`	TEXT	Download API URL and storage path for the ZIP
`liveUrl`	TEXT	Live/external URL (website, Figma, demo, etc.)
`images`	TEXT[]	Array of storage paths for uploaded images
`status`	TEXT	'Pending', 'Approved', or 'Rejected'
`uploadDate`	TIMESTAMPTZ	When the project was submitted
`downloadCount`	INTEGER	Total downloads (atomic counter)
`viewCount`	INTEGER	Total views (atomic counter, deduplicated by cookie)
`helpfulCount`	INTEGER	Number of "helpful" votes
`helpfulBy`	UUID[]	User IDs who voted helpful
`uploaderId`	UUID (FK)	References `users(userId)`

5.3 Downloads Table

Column	Type	Description
`downloadId`	UUID (PK)	Auto-generated
`userId`	UUID (FK, nullable)	Who downloaded
`projectId`	UUID (FK)	Which project
`downloadDate`	TIMESTAMPTZ	When

5.4 Bookmarks Table

Column	Type	Description

`bookmarkId`	UUID (PK)	Auto-generated
`userId`	UUID (FK)	Who bookmarked
`projectId`	UUID (FK)	Which project
`bookmarkDate`	TIMESTAMPTZ	When

A **UNIQUE constraint** on `(userId, projectId)` ensures a user can bookmark a project only once.

5.5 Notifications Table

Column	Type	Description
`notificationId`	UUID (PK)	Auto-generated
`userId`	UUID (FK)	Recipient user
`message`	TEXT	Notification content
`status`	TEXT	'unread' or 'read'
`dateCreated`	TIMESTAMPTZ	Creation timestamp

6. Authentication & Authorization

6.1 Authentication Flow

1. **Registration** (`app/register/page.tsx`): User fills the form, client calls `supabase.auth.signUp()` with `emailRedirectTo` pointing to `/login?confirmed=1` . A profile row is created in the `users` table via the `createUserProfile()` server action.
2. **Email Confirmation:** Supabase sends a confirmation email. User clicks the link to verify their account.
3. **Login** (`app/login/page.tsx`): Client calls `supabase.auth.signInWithPassword()` . On success, the `getCurrentUser()` server action fetches the user profile from the database.
4. **Session Management:** Supabase stores auth tokens in HTTP-only cookies via `@supabase/ssr` . Server Components read these cookies using `createAuthClient()` in `lib/supabase-server.ts` .
5. **Password Reset:** `forgot-password` page sends a reset email via `supabase.auth.resetPasswordForEmail()` . The `reset-password` page listens for the `PASSWORD_RECOVERY` auth event and allows updating the password.

6.2 Authorization Layers

Function	Location	Purpose
`getCurrentUser()`	`lib/auth.ts`	Returns the current user or `null` . Gracefully handles stale sessions.
`requireAuth()`	`lib/auth.ts`	Throws `Unauthorized` if no user is logged in.
`requireAdmin()`	`lib/auth.ts`	Throws `Forbidden` if user is not an admin.
`requireUploaderOrAdmin()`	`lib/auth.ts`	Allows access only to the project uploader or an admin.

6.3 Admin Secret Code

Admin registration requires a secret code validated server-side by `verifyAdminSecretCode()` . This prevents self-elevation - the `role` field is never read from `user_metadata` (which is client-settable).

6.4 Row Level Security

All five tables have RLS enabled. Key policies:

- **Users:** Can read own profile; admins read all. Users update own profile but cannot change their role.
- **Projects:** Anyone reads approved projects; uploaders/admins see pending/rejected.
- **Bookmarks:** Users manage only their own bookmarks.
- **Notifications:** Users read/update only their own notifications.

7. Screens & Pages

7a. Landing / Home Page

Property	Detail
File	`app/page.tsx` (Server Component)
URL	`/`
Description	The main entry point of the application. Shows a hero section with statistics (total projects, departments, downloads), a search bar for logged-in users, or a sign-up prompt for guests. Features sections for "In the Archive" (featured projects), "Browse by Department" (clickable department cards with project counts), "Recently Uploaded" projects, a "Start Contributing" CTA, and "How the archive works" info cards.
Guest View	Shows locked/blurred project cards (`LockedProjectCard`) encouraging registration. Search is not available.
Logged-in View	Full search bar, project listings, department browsing, quick-access links to Dashboard, Search, Upload, Profile.
Key Data	Calls `getRepositoryStats()` , `getApprovedProjects()` for recent and popular projects.

7b. Registration Page

Property	Detail
File	`app/register/page.tsx` (Client Component)
URL	`/register`
Description	Split-screen layout with a promotional panel on the left and the registration form on the right. Supports Student and Admin role toggle. Collects: full name, email, student index number, department (120+ departments from dropdown), programme, level/year, contact number (10-digit validation), password (min 6 chars), confirm password. Admin registration requires a secret code field.
Validation	Client-side validation for password match, length, required fields, 10-digit phone. Server-side duplicate email check via `checkEmailExists()` . Supabase also checks for existing identities.

Post-Submit	Shows "Check your email" confirmation with a link back to login.

7c. Login Page

Property	Detail
File	`app/login/page.tsx` (Client Component)
URL	`/login`
Description	Centered card with Student/Admin toggle tabs. Email + password fields with show/hide toggle. Admin login requires an additional "Admin Secret Code" field. Shows green banners for "Account created successfully" (after registration) and "Email confirmed" (after email verification). Includes "Forgot Password?" link and "Remember this device" checkbox. Redirects to `/admin` for admins, `/dashboard` for students (or returnUrl).

7d. Forgot Password

Property	Detail
File	`app/forgot-password/page.tsx` (Client Component)
URL	`/forgot-password`
Description	Simple email input form. Calls `supabase.auth.resetPasswordForEmail()` with `redirectTo` set to `{origin}/reset-password`. Shows success/error messages.

7e. Reset Password

Property	Detail
File	`app/reset-password/page.tsx` (Client Component)
URL	`/reset-password`
Description	Activated from the email reset link. Listens for Supabase `PASSWORD_RECOVERY` auth event. Shows "Verifying your reset link..." while waiting. Once verified, shows new password + confirm password form. Calls `supabase.auth.updateUser({ password })`. Auto-redirects to login after success. 15-second timeout for invalid/expired links.

7f. Student Dashboard

Property	Detail
File	`app/dashboard/page.tsx` (Server Component)
URL	`/dashboard`
Layout	`app/dashboard/layout.tsx` - includes `DashboardSidebar` with navigation links
Description	Welcome message with user's name. Four stat cards: **Repository Total** (with "+X this week"), **My Uploads** (with pending count), **Bookmarks**, **Recent Downloads**. **Recent Activity** timeline showing last 3 uploads. **Notifications** panel with unread count badge. "My Uploads" table with title, department, year, status badges, downloads, views. CTA for searching projects. "Pro Tip" box about keywords.
Key Data	Calls `getMyProjects()`, `getStudentStats()`, `getMyNotifications()`, `getRepositoryStats()` in parallel via `Promise.all()`.

7g. Upload Project

Property	Detail
File	`app/dashboard/upload/page.tsx` (Client Component)
URL	`/dashboard/upload`
Description	Form with fields: Title, Author Name, Department (searchable datalist with 120+ options), Academic Year, Project Type (dropdown: Research Paper, Software/App, Website, Design/Art, Other), Abstract (textarea), Keywords (comma-separated). **Artifacts section:** GitHub URL, Live/External URL, PDF upload (max 50MB), ZIP upload (max 50MB), Images (up to 5 images, max 10MB each, JPEG/PNG/WEBP/GIF). At least one artifact is required. Validates file types and sizes. Submits as `FormData` to `uploadProject()` server action. Projects start as "Pending" and admins are notified.

7h. Upload History

Property	Detail
File	`app/dashboard/history/page.tsx` (Client Component)
URL	`/dashboard/history`
Description	Table of all user's uploads with columns: Title (clickable link), Department, Year, Status (badge), Uploaded date, Downloads, Views. Includes search-by-title and status filter (All/Pending/Approved/Rejected). "Submit New Project" button.

7i. Bookmarks

Property	Detail
File	`app/dashboard/bookmarks/page.tsx` (Server Component)
URL	`/dashboard/bookmarks`
Description	Grid of bookmarked projects displayed as `ProjectCard` components. Each card has an "unbookmark" button overlay. Empty state with "Browse projects" link.

7j. Profile Settings

Property	Detail
File	`app/dashboard/profile/page.tsx` (Client Component)
URL	`/dashboard/profile`
Description	**Profile Photo** section with avatar upload (camera icon overlay, JPEG/PNG/WEBP, max 5MB). Shows initials if no avatar. **Personal Information** form: Full Name, Email (read-only), Department, Contact (10-digit validation). **Academic Credentials** section: Index Number, Programme (dropdown), Level/Year (dropdown). Save button calls `updateProfile()` server action.

7k. Projects Search / Browse

Property	Detail
File	`app/projects/page.tsx` (Client Component)
URL	`/projects`
Description	Two-column layout: **Filter sidebar** (hidden on mobile, toggle button) with Quick Search input, Department radio buttons (6 main + "Other" with searchable datalist of 120+ departments), Project Type radio buttons, Academic Year slider, Popular Keywords chips (Blockchain, AI, ML, Cloud, Cybersecurity, IoT), Clear All button. **Results area** showing project count, sort dropdown (Newest/Most Downloaded/Department), list of `ProjectListItem` cards with year badge, department, keywords, title, author, abstract, download/view counts, "View Details" and "Download PDF" buttons. Search is debounced (350ms) and filters reflect in URL params.

7l. Project Detail

Property	Detail
File	`app/projects/[id]/page.tsx` (Server) + `components/ProjectDetailClient.tsx` (Client)
URL	`/projects/[id]`
Description	Full project detail view with: title, author, department, academic year, status badge, artifact badges (PDF/ZIP/GitHub/Live URL/Images), abstract, keywords, download/view/helpful counts. Action buttons: Download PDF, Download ZIP (if available), View on GitHub (opens `GitHubRepoViewer`), Visit Live URL, Bookmark toggle, "Mark as Helpful" toggle. Image gallery with signed URLs. Related projects section (up to 3 from same department). Records views (deduplicated by cookie).

7m. Admin Dashboard

Property	Detail
File	`app/admin/page.tsx` (Client Component)
URL	`/admin`
Description	Three stat cards: Total Projects, Registered Users, Total Downloads. **Pending Review Queue** listing pending projects with title, author, department, year, abstract, status badge, artifact badges. Each item has View, Approve (green), Reject (red) action buttons. Empty state for no pending projects.

7n. Admin Projects Management

Property	Detail
File	`app/admin/projects/page.tsx` (Client Component)
URL	`/admin/projects`
Description	Full table of all projects (any status). Status filter dropdown (All/Pending/Approved/Rejected). Columns: Title (clickable), Artifacts, Author, Department, Year, Status badge. Actions per row: View, Approve (if not approved), Reject (if not rejected), Delete (with confirmation dialog). Calls `getAllProjects()`, `approveOrRejectProject()`, `deleteProject()`.

7o. Admin Users Management

Property	Detail
File	`app/admin/users/page.tsx` (Client Component)
URL	`/admin/users`
Description	Searchable user table (debounced 300ms, searches name, email, department). Columns: Name, Email, Department, Role (badge with icon), Status (Active/Deactivated badge), Joined date, Actions. Deactivate button with confirmation dialog. Calls `listUsers()`, `deactivateUser()`.

7p. Admin Statistics

Property	Detail

File	`app/admin/statistics/page.tsx` (Client Component)
URL	`/admin/statistics`
Description	Four stat cards: Total Projects, Total Users, Total Downloads, Avg Downloads per project. Two Recharts bar charts: **"Uploads per Month"** (vertical bars by month) and **"Top Departments"** (horizontal bars by department name). Calls `getProjectStats()`.

8. Communication & Data Flow

8.1 Client-to-Server Communication

Method	How It Works	Example
Server Components	Pages marked without `'use client'` run on the server. They call async functions directly (no API calls needed).	`app/page.tsx` calls `getApprovedProjects()` and `getRepositoryStats()` directly during render.
Server Actions	Functions marked `'use server'` in `lib/actions/`. Client Components import and call them as regular async functions. Next.js handles the HTTP serialization.	`uploadProject(formData)` is called from the upload form. `approveOrRejectProject(id, status)` is called from admin buttons.
API Routes	RESTful endpoints in `app/api/`. Used where browser redirects or direct HTTP responses are needed.	`GET /api/download/[id]?type=pdf` generates a signed URL and 302-redirects the browser.
Client-Side Supabase	Auth operations (signUp, signIn, signOut, resetPassword) use the browser Supabase client directly.	`supabase.auth.signInWithPassword({ email, password })`

8.2 Server-to-Supabase Communication

Two Supabase clients are used on the server (`lib/supabase-server.ts`):

Client	Purpose	Key
`supabaseAdmin`	Service-role client for all database reads/writes. Bypasses Row Level Security.	`SUPABASE_SERVICE_ROLE_KEY`
`createAuthClient()`	Per-request client that reads the user's auth cookie. Used to	`NEXT_PUBLIC_SUPABASE_ANON_KEY`

	identify who is logged in.	

8.3 Key Data Flow: Project Upload

1. Student fills upload form (Client Component)
2. Form submits `FormData` to `uploadProject()` Server Action
3. Server Action validates fields and file sizes/types
4. Files (PDF, ZIP, images) uploaded to Supabase Storage bucket `"projects"`
5. Project record inserted into `projects` table with status `"Pending"`
6. Notification created for all admin users via `createNotification()`
7. Student redirected to dashboard

8.4 Key Data Flow: Project Download

1. User clicks "Download PDF" link, which points to `/api/download/[id]?type=pdf`
2. API route verifies authentication and authorization
3. Generates a 1-hour signed URL from Supabase Storage
4. Records the download in the `downloads` table and increments `downloadCount`
5. 302-redirects the browser to the signed URL for direct file download

9. Notification System

Code Location: `lib/actions/notifications.ts`, `components/NotificationBell.tsx`

Notification Triggers

Event	Recipients	Message
New project uploaded	All admin users	"New project pending review: {title}"
Project approved	The uploader	"Your project '{title}' has been approved"
Project rejected	The uploader	"Your project '{title}' has been rejected"

Notification Bell Component

The `NotificationBell` component in the header shows a red badge with the count of unread notifications. Clicking it opens a dropdown with recent notifications. The "Mark all read" button calls `markAllNotificationsRead()`.

10. File Storage & Downloads

Storage Architecture

- **Bucket:** `"projects"` (private, created via `schema.sql`)
- **File Paths:**
  - PDFs: `projects/{projectId}/document.pdf`
  - ZIPs: `projects/{projectId}/source.zip`
  - Images: `projects/{projectId}/images/0.jpg`, `1.png`, etc.
- **Avatar Bucket:** `"avatars"` (separate bucket for profile photos)
- **Access Control:** All files are uploaded and served via the service-role key. Signed URLs (1-hour expiry) are generated for downloads and image display.
- **Size Limits:** PDF/ZIP: 50MB, Images: 10MB each (max 5), Avatars: 5MB

Download API

Endpoint: `GET /api/download/[id]?type=pdf|zip` (`app/api/download/[id]/route.ts`)

Supports a `?preview` query param for in-browser PDF viewing (no download header, no download count recorded).

11. PWA (Progressive Web App)

Configuration

File	Purpose
`public/manifest.json`	PWA manifest with app name, icons, theme color, start URL, display mode
`public/sw.js`	Service worker for caching and offline support
`public/icons/icon-512x512.png`	Full-bleed 512x512 icon (maskable)
`public/icons/icon-192x192.png`	Full-bleed 192x192 icon
`components/PWAProvider.tsx`	Client component that registers the service worker on mount
`app/layout.tsx`	Includes manifest link and Apple Web App meta tags in metadata

Installation

Users can install the app from Chrome (desktop or mobile) via the address bar install icon or "Add to Home Screen" on mobile. The Vercel deployment uses HTTPS, enabling seamless PWA installation.

12. Security Measures

Measure	Implementation
Authentication	Supabase Auth with email/password. Sessions stored in HTTP-only cookies. Automatic token refresh via `@supabase/ssr`.
Admin Protection	Admin role requires a server-validated secret code. Role is never read from client-settable `user_metadata` to prevent privilege escalation.
Row Level Security	All 5 database tables have RLS policies. Even if someone bypasses the app, direct database access is restricted.
Service Role Isolation	The service-role key (`SUPABASE_SERVICE_ROLE_KEY`) is only used server-side. The browser only has access to the anon key.
File Access Control	Storage bucket is private. Files are only accessible via time-limited signed URLs generated server-side after authorization checks.
Input Validation	Both client-side and server-side validation for all forms. File type checking, size limits, required field enforcement.
CSRF Protection	Next.js Server Actions include built-in CSRF protection via origin verification.
Environment Variables	Secrets stored in `.env.local` (gitignored). Only `NEXT_PUBLIC_*` vars are exposed to the browser.

13. Deployment

Property	Detail
Platform	Vercel (serverless)
Live URL	`https://miniproject-one-steel.vercel.app/`
Build Command	`next build`
Dev Command	`next dev`
Node.js	20.x
Environment Variables	Set in Vercel dashboard: `NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` , `SUPABASE_SERVICE_ROLE_KEY` , `ADMIN_SECRET_CODE`

Supabase Configuration

- **Site URL:** `https://miniproject-one-steel.vercel.app/`
- **Redirect URLs:** Include both the Vercel URL and `http://localhost:3000` for local development
- **Email Templates:** Confirmation and password reset emails configured in Supabase dashboard

14. Shared Components

Component	File	Description
`Header`	`components/Header.tsx`	Global sticky navigation bar. Logo + app name, navigation links (Home, Browse Projects, Departments, About), search input, notification bell, dark/light theme toggle, "New Project" link, user profile dropdown (with avatar, info, dashboard/profile/logout links), mobile hamburger menu.
`DashboardSidebar`	`components/DashboardSidebar.tsx`	Side navigation for both student and admin dashboards. Shows role-appropriate links with active state highlighting. Student links: Dashboard, My Uploads, Bookmarks, Profile. Admin links: Dashboard, All Projects, Users, Statistics.
`AuthProvider`	`components/AuthProvider.tsx`	React Context provider for user authentication state. Provides `user`, `setUser`, and `logout` across all components. Hydrated from server-side `getCurrentUser()` on initial load.
`ThemeProvider`	`components/ThemeProvider.tsx`	React Context for dark/light theme management. Persists

		preference in a cookie. Toggles `dark` class on `<html>`.
`ProjectCard`	`components/ProjectCard.tsx`	Card component for grid display of projects. Shows title, author, department, year, abstract (truncated), keyword badges, download/view/helpful counts, artifact badges.
`ProjectDetailClient`	`components/ProjectDetailClient.tsx`	Full project detail view with all metadata, download buttons, bookmark/helpful toggles, GitHub repo viewer, image gallery, related projects section.
`ProjectStatusBadge`	`components/ProjectStatusBadge.tsx`	Colored badge component: green for Approved, yellow for Pending, red for Rejected.
`ProjectArtifactBadges`	`components/ProjectArtifactBadges.tsx`	Shows which artifacts a project has: PDF, ZIP, GitHub, Live URL, Images - as small icon badges.
`NotificationBell`	`components/NotificationBell.tsx`	Bell icon in the header with unread count badge and dropdown notification list.
`GitHubRepoViewer`	`components/GitHubRepoViewer.tsx`	Fetches and displays the file tree of a GitHub repository using the GitHub API.
`LockedProjectCard`	`components/LockedProjectCard.tsx`	Blurred placeholder card shown to guests, encouraging them to create an account.

`PWAProvider`	`components/PWAProvider.tsx`	Registers the service worker on component mount for PWA support.

15. Summary of Key Files

Quick reference for locating important code:

Category	Files
Authentication Logic	`lib/auth.ts`, `lib/actions/auth.ts`, `lib/supabase-server.ts`, `lib/supabase-client.ts`
Project CRUD	`lib/actions/projects.ts` (upload, search, approve, delete, bookmarks, downloads, stats)
Notifications	`lib/actions/notifications.ts`, `components/NotificationBell.tsx`
Type Definitions	`lib/types.ts` (User, Project, Download, Bookmark, Notification)
Constants	`lib/constants.ts` (120+ departments, programmes, levels, keywords, project types)
Database Schema	`supabase/schema.sql`, `supabase/atomic_counters.sql`, `supabase/project_types_and_media.sql`
File Downloads	`app/api/download/[id]/route.ts`
Global Layout	`app/layout.tsx` (root), `app/dashboard/layout.tsx` (student), `app/admin/layout.tsx` (admin)
Navigation	`components/Header.tsx`, `components/DashboardSidebar.tsx`
Styling	`app/globals.css` (Tailwind imports + custom animations: fade-in, marquee, float)
PWA	`public/manifest.json`, `public/sw.js`, `components/PWAProvider.tsx`
Configuration	`package.json`, `next.config.ts`, `tsconfig.json`, `.env.local`

Students Academic Repository — Site Documentation

Generated September 2026 · Next.js 16 · React 19 · Supabase · Tailwind CSS 4 · Vercel